



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



UNIVERSIDAD TÉCNICA DE AMBATO

Facultad de Ingeniería en Sistemas, Electrónica e Industrial

Título:	APE GPIS 3: Elaboración de un Manual de Pruebas con las Herramientas Seleccionadas
Carrera:	Software
Unidad de Organización Curricular:	Profesional
Nivel y Paralelo:	Sexto - A
Alumno:	Carrasco Paredes Kevin Andres Chimborazo Guaman William Andres Loor Sandoya Jins Steeven Quishpe Lopez Luis Alexander Sailema Gavilanez Ismael Alexander
Módulo y Docente:	Gestión de Pruebas e Implantación del Software
Docente:	Ing. Leonardo David Torres Valverde



**UNIVERSIDAD
TÉCNICA DE AMBATO**



INFORME DE GUÍA PRÁCTICA

I. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar un manual sobre la realización de pruebas con las herramientas seleccionadas

Específicos:

- Investigar información detallada sobre las herramientas elegidas para pruebas funcionales, no funcionales, análisis de código estático con su respectiva configuración.
- Implementar y desarrollar cada una de las pruebas requeridas por el docente dentro del proyecto de Seguros asignado por el docente.
- Documentar el desarrollo de la implementación de cada una de las pruebas describiendo el paso a paso con el fin de realizar una guía completa y útil.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 4

No presenciales: 0

2.4 Instrucciones

En base a las herramientas seleccionadas para realizar pruebas funcionales, no funcionales y análisis de código estático, los estudiantes deberán elaborar un manual que explique detalladamente cómo realizar las pruebas con dichas herramientas. El manual debe incluir las siguientes secciones: Instalación y configuración de las herramientas: Paso a paso de cómo instalar y configurar las herramientas seleccionadas en los diferentes entornos de desarrollo, como local, CI/CD, y producción, según corresponda. Configuración de entornos de prueba: Descripción de cómo preparar los entornos para ejecutar las pruebas (Ej. configuración de entornos de prueba automatizada, gestión de dependencias, integración continua). Ejemplos de pruebas: Proporcionar ejemplos prácticos de pruebas funcionales, no funcionales y análisis de código estático realizadas con las herramientas seleccionadas. Cada ejemplo debe incluir el código, los resultados esperados y la interpretación de dichos resultados. Solución de problemas comunes: Incluir una sección con los errores más comunes que pueden surgir durante la instalación, configuración o ejecución de las pruebas, junto con sus soluciones. El manual debe ser presentado en formato PDF y debe estar estructurado de manera clara, con capturas de pantalla y ejemplos que faciliten su comprensión.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Internet
- Herramientas de pruebas seleccionadas
- Entornos de desarrollo

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☐ Gamificación



☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

En base a las herramientas seleccionadas para realizar pruebas funcionales, no funcionales y análisis de código estático, los estudiantes deberán elaborar un manual que explique detalladamente cómo realizar las pruebas con dichas herramientas. El manual debe incluir las siguientes secciones: Instalación y configuración de las herramientas: Paso a paso de cómo instalar y configurar las herramientas seleccionadas en los diferentes entornos de desarrollo, como local, CI/CD, y producción, según corresponda. Configuración de entornos de prueba: Descripción de cómo preparar los entornos para ejecutar las pruebas (Ej. configuración de entornos de prueba automatizada, gestión de dependencias, integración continua). Ejemplos de pruebas: Proporcionar ejemplos prácticos de pruebas funcionales, no funcionales y análisis de código estático realizadas con las herramientas seleccionadas. Cada ejemplo debe incluir el código, los resultados esperados y la interpretación de dichos resultados. Solución de problemas comunes: Incluir una sección con los errores más comunes que pueden surgir durante la instalación, configuración o ejecución de las pruebas, junto con sus soluciones. El manual debe ser presentado en formato PDF y debe estar estructurado de manera clara, con capturas de pantalla y ejemplos que faciliten su comprensión.

2.7 Resultados obtenidos

BACK-END

- Pruebas Funcionales

Las pruebas funcionales se centran en verificar que cada componente del sistema cumpla correctamente con los requisitos definidos, evaluando el comportamiento esperado ante distintos escenarios de uso. En este trabajo se emplearon **JUnit** y **Mockito** como herramientas principales para automatizar estas pruebas dentro del entorno Spring Boot.

- **JUnit** permite ejecutar pruebas unitarias e integradas para validar la lógica de negocio, controladores y servicios.
- **Mockito** facilita la creación de objetos simulados (mocks) para aislar dependencias y asegurar que los métodos se comporten correctamente incluso sin conexión a bases de datos u otros servicios.

Pruebas Unitarias y de integración

Instalación y configuración de JUnit

Para realizar las pruebas unitarias e integración en un proyecto de Spring Boot, se utiliza JUnit como motor principal de pruebas. Spring Boot facilita esta integración a través de su dependencia oficial para testing.

Pasos para instalar y configurar:

1. Abrir el proyecto de Spring Boot en un IDE compatible como IntelliJ IDEA.
2. Asegurarse de tener configurado el sistema de construcción Maven o Gradle.
3. Agregar en el archivo de dependencias (pom.xml para Maven) la librería spring-boot-starter-test.
 - a. Esta dependencia incluye JUnit 5, Mockito y herramientas adicionales de prueba.



4. Confirmar que la dependencia esté instalada correctamente haciendo una actualización del proyecto.
5. Verificar que las carpetas de prueba estén estructuradas dentro del proyecto en src/test/java.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
4     <modelVersion>4.0.0</modelVersion>
5     <groupId>com.example</groupId>
6     <artifactId>aplicacion-seguros</artifactId>
7     <version>1.0.0</version>
8     <packaging>jar</packaging>
9     <name>aplicacion-seguros</name>
10    <description>Aplicación de seguros</description>
11    <dependencies>
12        <dependency>
13            <groupId>org.springframework.security</groupId>
14            <artifactId>spring-security-test</artifactId>
15            <scope>test</scope>
16        </dependency>
17        <dependency>
18            <groupId>org.junit.jupiter</groupId>
19            <artifactId>junit-jupiter</artifactId>
20            <version>5.9.1</version>
21            <scope>test</scope>
22        </dependency>
23        <dependency>
24            <groupId>com.h2database</groupId>
25            <artifactId>h2</artifactId>
26            <version>2.3.232</version>
27            <scope>test</scope>
28        </dependency>
29    </dependencies>
30    </project>
```

Ilustración 1 Inserción de dependencias de JUnit y H2 en pom.xml

Configuración de entornos de prueba

Para garantizar la correcta ejecución de las pruebas, se prepararon dos tipos de entornos:

Entorno de pruebas unitarias

Se trabajó sin cargar el contexto completo de Spring. Se probaron clases específicas de manera aislada, utilizando herramientas como Mockito para simular las dependencias externas.

Entorno de pruebas de integración

Se configuró para levantar el contexto completo de Spring Boot, simulando el comportamiento real de la aplicación. Para las pruebas de integración, se utilizó una base de datos H2 en memoria, lo que permitió realizar operaciones de persistencia sin afectar bases de datos reales.

Pruebas Unitarias

Cómo hacerlas:

1. Crear una clase de prueba en src/test/java siguiendo la misma estructura de paquetes del proyecto.



```
package com.app.seguros.AplicacionSeguros.Services;  
  
> import ...  
  
class UsuarioServiceTest { no usages new *  
  
}
```

Ilustración 2 Clase de prueba para usuarios

2. Anotar cada método de prueba con @Test.

```
class UsuarioServiceTest { new *  
  
    @Test new *  
    void getUsers() {  
    }  
  
    @Test new *  
    void getUserByEmail() {  
    }  
  
    @Test new *  
    void getUserById() {  
    }  
  
    @Test new *  
    void register() {  
    }  
}
```

Ilustración 3 Creacion de Metodos de prueba

3. Instanciar directamente el objeto a probar o utilizar herramientas como Mockito para simular dependencias si es necesario.

```
@ExtendWith(MockitoExtension.class) Jostero32  
class UsuarioServiceUnitariasTest {  
  
    @Mock 16 usages  
    private UsuariosRepository usuariosRepository;  
  
    @Mock 2 usages  
    private PasswordEncoder passwordEncoder;  
  
    @InjectMocks 8 usages  
    private UsuarioService usuarioService;  
  
    private Usuario create(int id) { ... }
```

Ilustración 4 Inyección de dependencias de UsuariosService



- Validar los resultados esperados mediante afirmaciones (asserts), como assertEquals, assertTrue, etc.

```
class UsuarioServiceUnitariasTest {  
    @Test & Jostero32  
    void getUsuarios() {  
        List<Usuario> usuarios=List.of(create(1),create(2));  
        when(usuariosRepository.findAll()).thenReturn(usuarios);  
  
        assertEquals(usuarioService.getUsuarios().size(), actual: 2);  
        verify(usuariosRepository,times( wantedNumberOfInvocations: 1)).findAll();  
    }  
  
    @Test & Jostero32  
    void getUsuarioByEmail() {  
        Usuario u=create(2);  
        when(usuariosRepository.findByEmail(any(String.class))).thenReturn(Optional.of(u));  
  
        assertEquals(usuarioService.getUsuarioByEmail("email").getPassword(), actual: "contraseña");  
        verify(usuariosRepository,times( wantedNumberOfInvocations: 1)).findByEmail(any(String.class));  
    }  
}
```

Ilustración 5 implementación de métodos de Prueba

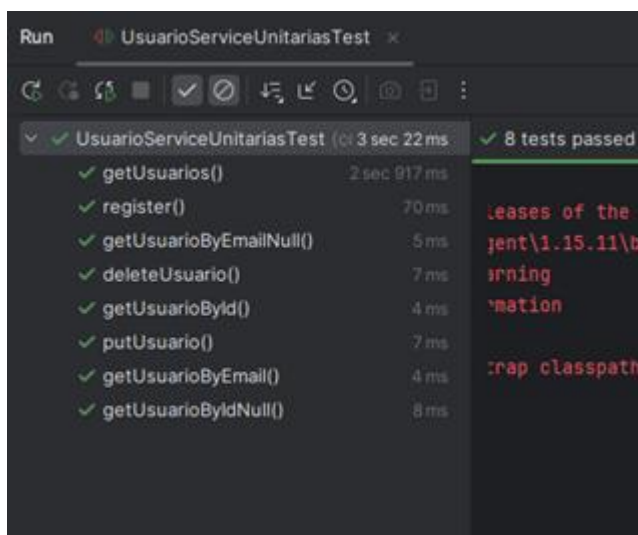


Ilustración 6 Ejecución de las pruebas

Pruebas de Integración

Cómo hacerlas:

- Crear una clase de prueba en la carpeta src/test/java, respetando la estructura del proyecto.



```
package com.app.seguros.AplicacionSeguros.Controllers;

import ...

class UsuariosControllerIntegracionTest { no usages Jostero32 *
    *
}

}
```

Ilustración 7 creación de clase de pruebas de integración

2. Anotar la clase de prueba con `@SpringBootTest` para levantar el contexto completo de Spring Boot.

```
package com.app.seguros.AplicacionSeguros.Controllers;

import ...

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.MOCK) no usages Jostero32 *
@AutoConfigureMockMvc
@TestPropertySource(locations = "classpath:application-test.properties")
class UsuariosControllerIntegracionTest {
    *
}

}
```

Ilustración 8 Anotaciones de la clase para prueba springboot y de properties para H2

3. Inyectar los componentes necesarios mediante `@Autowired`.

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.MOCK) no usages Jostero32 *
@AutoConfigureMockMvc
@TestPropertySource(locations = "classpath:application-test.properties")
class UsuariosControllerIntegracionTest {

    @Autowired no usages
    private MockMvc mockMvc;

    @Autowired no usages
    private UsuariosRepository usuariosRepository;

    @Autowired no usages
    private PasswordEncoder passwordEncoder;

    private ObjectMapper objectMapper = new ObjectMapper(); no usages

    private static Usuario create(int id){
        *
    }

}
```

Ilustración 9 Inyección de componentes

4. Ejecutar operaciones que involucren flujos completos, como inserciones y consultas de datos.



```
@Test & JUnit5
void postUsuario() throws Exception{
    UsuarioCrearRequest uGuardar=new UsuarioCrearRequest( email: "emailGuardar", password: "contraseña", nombre:
    mockMvc.perform(post( uriTemplate: "/Usuarios/guardar")
        .header( name: "Authorization", values: "Bearer "+token)
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(uGuardar)))
        .andExpect(status().isCreated())
        .andExpect(jsonPath( expression: "$.email").value( expectedValue: "emailGuardar"));
}

@Test & JUnit5
void postUsuarioNull() throws Exception{
    mockMvc.perform(post( uriTemplate: "/Usuarios/guardar")
        .header( name: "Authorization", values: "Bearer "+token)
        .contentType(MediaType.APPLICATION_JSON)
        .content(""))
        .andExpect(status().isBadRequest());
}
```

Ilustración 10 Implementación de métodos de prueba

5. Configurar la aplicación para que utilice una **base de datos H2 en memoria** durante las pruebas, asegurando que los datos persistan únicamente mientras dure la prueba.

```
@BeforeEach & JUnit5
void setUp(){
    usuariosRepository.deleteAll();
    Usuario u= Usuario.builder()
        .rol(Rol.Administrador)
        .nombre("nombre")
        .apellido("apellido")
        .estado(Estado.Activo)
        .email("email")
        .contraseña(passwordEncoder.encode( rawPassword: "contraseña"))
        .build();
    idGenerado=usuariosRepository.save(u).getId_Usuario();

    Usuario u1= Usuario.builder()
        .rol(Rol.Administrador)
        .nombre("nombre1")
        .apellido("apellido1")
        .estado(Estado.Activo)
        .email("email1")
        .contraseña(passwordEncoder.encode( rawPassword: "contraseña1"))
        .build();
    usuariosRepository.save(u1);
    JwtService jwtService=new JwtService();
    token= jwtService.getToken(u);
}
```

Ilustración 11 Configuración de H2 para datos antes de cada prueba

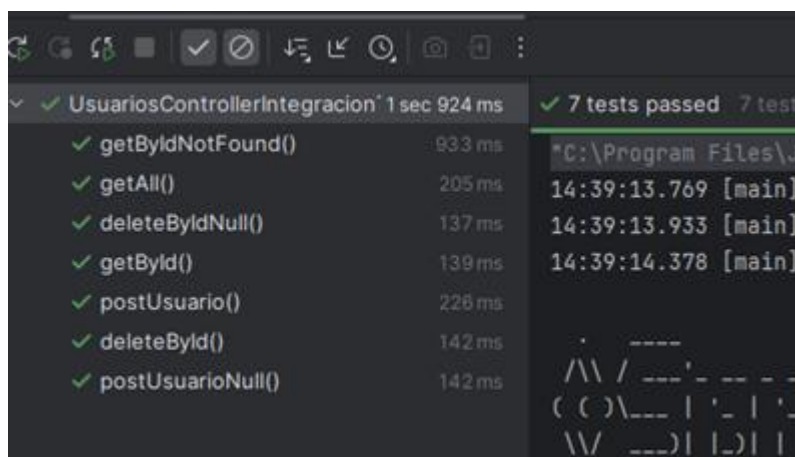


Ilustración 12 Ejecución de Pruebas

Pruebas No Funcionales

Las pruebas no funcionales permiten evaluar características de calidad del sistema más allá de su funcionalidad básica. En este trabajo se aplican dos tipos clave:

- **Pruebas de carga y rendimiento** mediante Apache JMeter, para medir el comportamiento del sistema bajo distintas condiciones de tráfico.
- **Pruebas de seguridad** con OWASP ZAP, enfocadas en identificar posibles vulnerabilidades en los endpoints expuestos. Estas pruebas son fundamentales para garantizar la estabilidad, eficiencia y protección del sistema en ambientes reales de uso.

Pruebas de carga y rendimiento

Descarga e instalación de Apache JMeter

- Acceder al sitio web oficial: <https://jmeter.apache.org/>.
- Descargar la última versión disponible de Apache JMeter en formato .zip.
- Extraer el contenido descargado en una carpeta de preferencia.
- Ejecutar JMeter:
 - En Windows: abrir el archivo ApacheJMeter.jar.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



Download Apache JMeter

We recommend you use a mirror to download our release builds, but you **must** [verify the integrity](#) of the downloaded files using signatures downloaded from our main distribution directories. Recent releases (48 hours) may not yet be available from all the mirrors.

You are currently using <https://d1c0n.apache.org/>. If you encounter a problem with this mirror, please select another mirror. If all mirrors are failing, there are backup mirrors (at the end of the mirrors list) that should be available.

Other mirrors: <https://d1c0n.apache.org/> [Change](#)

The [KEYS](#) link links to the code signing keys used to sign the product. The [PGP](#) link downloads the OpenPGP compatible signature from our main site. The [SHA-512](#) link downloads the sha512 checksum from the main site. Please [verify the integrity](#) of the downloaded file.

For more information concerning Apache JMeter, see the [Apache JMeter](#) site.

[KEYS](#)

Apache JMeter 5.6.3 (Requires Java 8+)

Binaries

[apache-jmeter-5.6.3.jar sha512 pgp](#)
[apache-jmeter-5.6.3.exe sha512 pgp](#)

Ilustración 13 Página Oficial de Jmeter

Extrayendo de apache-jmeter-5.6.3.zip

C:\Users\User\Desktop\apache-jmeter-5.6.3.zip
extrayendo
..travis.yml 100%

Tiempo transcurrido 00:00:00

Procesado 1%

[Segundo plano](#) [Pausa](#)
[Cancelar](#) [Modo...](#) [Ayuda](#)

Ilustración 14 Extraccion del programa descargado

Nombre	Fecha de modificación	Tipo	Tamaño
examples	05/05/2023 01:21 p. m.	Carpeta de archivos	
report-template	05/05/2023 01:21 p. m.	Carpeta de archivos	
templates	05/05/2023 01:21 p. m.	Carpeta de archivos	
ApacheJMeter.jar	02/01/2024 04:42 p. m.	Ejecutable (ar File	14 KB
BeanShellAssertion.bshrc	05/05/2023 01:21 p. m.	Archivo BSHRC	2 KB
BeanShellFunction.bshrc	05/05/2023 01:21 p. m.	Archivo BSHRC	3 KB
BeanShellListeners.bshrc	05/05/2023 01:21 p. m.	Archivo BSHRC	2 KB
BeanShellSamplers.bshrc	05/05/2023 01:21 p. m.	Archivo BSHRC	3 KB
create-mni-keystore.bat	05/05/2023 01:21 p. m.	Archivo por lotes ...	2 KB
create-mni-keystore.sh	05/05/2023 01:21 p. m.	Shell Script	2 KB
ic-parametern	05/05/2023 01:21 p. m.	Archivo PARAMET...	2 KB
heapdump.cmd	05/05/2023 01:21 p. m.	Script de comand...	2 KB
heapdump.sh	05/05/2023 01:21 p. m.	Shell Script	1 KB
jmeter.conf	05/05/2023 01:21 p. m.	Archivo CONF	2 KB
jmeter.jar	05/05/2023 01:21 p. m.	Archivo	9 KB
jmeter.bat	05/05/2023 01:21 p. m.	Archivo por lotes ...	9 KB
jmeter.log	28/04/2025 12:11 a. m.	Documento de te...	0 KB
jmeter.properties	04/07/2023 10:41 a. m.	Archivo de origen...	10 KB
jmeter.sh	05/05/2023 01:21 p. m.	Shell Script	5 KB
jmeter.n.cmd	05/05/2023 01:23 p. m.	Script de comand...	2 KB
jmeter.n.cmd	05/05/2023 01:23 p. m.	Script de comand...	2 KB
jmeter.server	05/05/2023 01:21 p. m.	Archivo	2 KB
jmeter.server.bat	05/05/2023 01:21 p. m.	Archivo por lotes ...	3 KB
jmeter.t.cmd	05/05/2023 01:21 p. m.	Script de comand...	2 KB
jmeter.t.cmd	05/05/2023 01:21 p. m.	Script de comand...	1 KB
log4j2.xml	05/05/2023 01:21 p. m.	Archivo de origen...	3 KB

Ilustración 15 Ejecucion de ApacheJMeter.jar



Pruebas de Rendimiento y Carga

Creación de un Plan de Pruebas

- En la ventana principal de JMeter, realizar clic derecho en el panel izquierdo.
- Seleccionar **Agregar** → **Plan de prueba**.
- Opcionalmente, asignar un nombre descriptivo al plan.

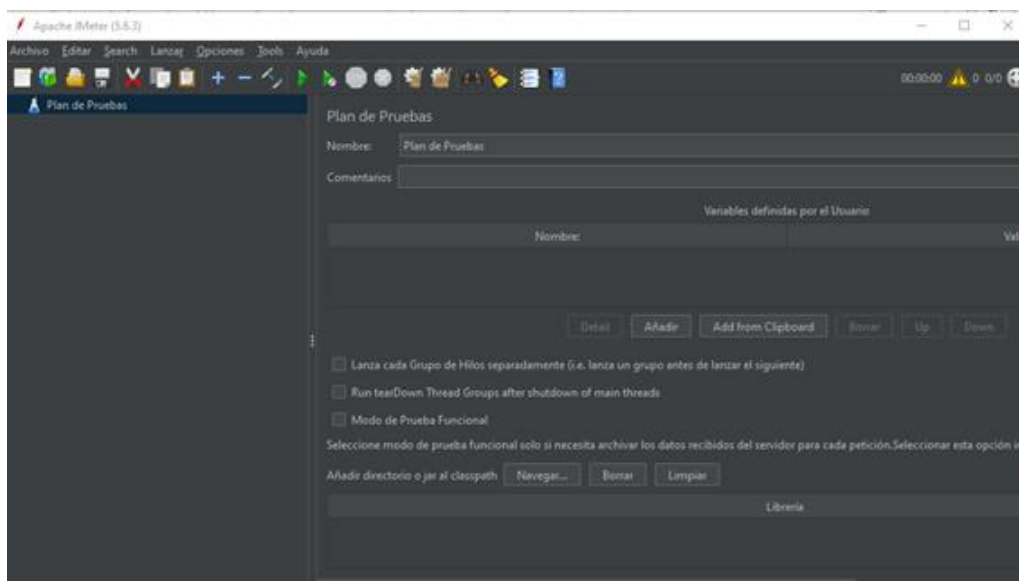


Ilustración 16 Vista de plan de pruebas

Configuración de Grupo de Hilos

- Clic derecho sobre el Plan de Prueba creado.
- Seleccionar **Agregar** → **Threads (Usuarios)** → **Grupo de Hilos**.
- Configurar los siguientes parámetros:
 - Número de hilos (usuarios): 1.
 - Período de subida (ramp-up): 1 segundo.
 - Número de repeticiones: 1.

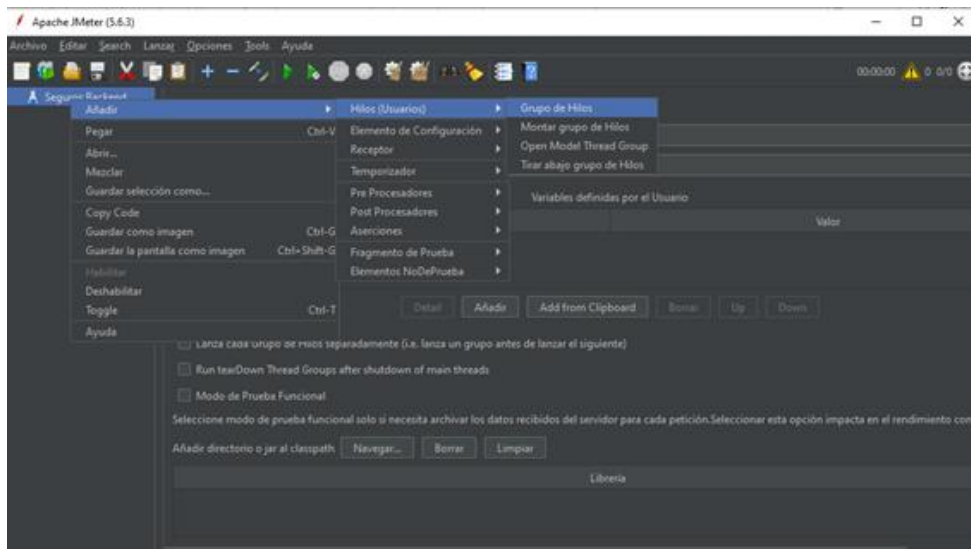


Ilustración 17 Inserción de Grupo de Hilos



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025

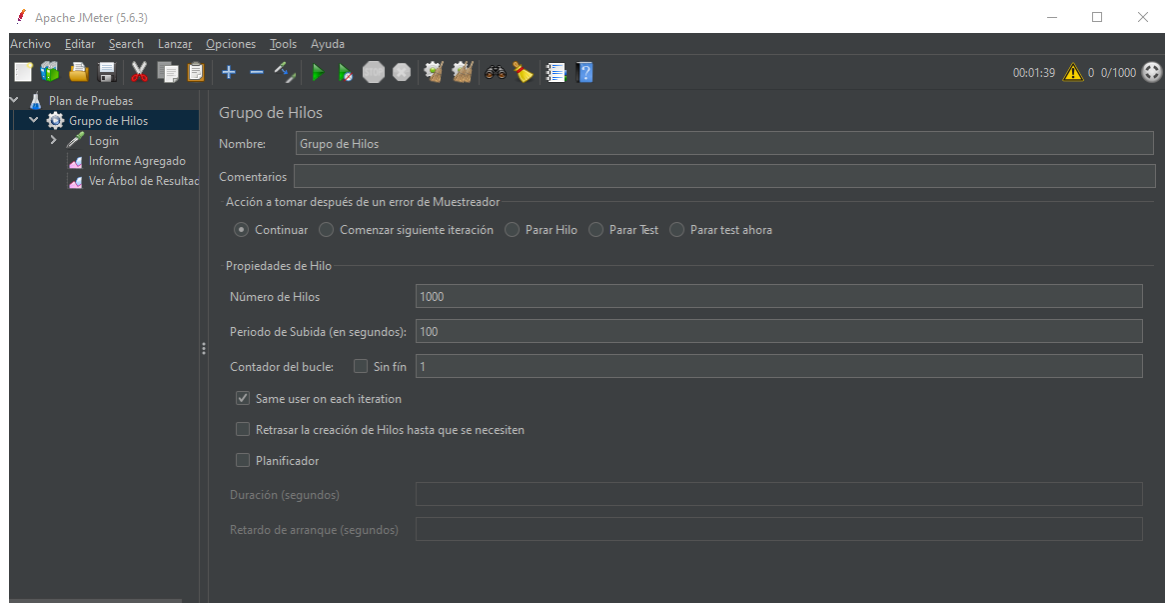


Ilustración 18 configuración de concurrencia

Creación de un HTTP Request (Solicitud HTTP POST)

- Clic derecho sobre el **Grupo de Hilos**.
- Seleccionar **Agregar** → **Sampler** → **HTTP Request**.
- Configurar:
 - Nombre del servidor o IP: localhost.
 - Puerto: 8080 (o el que utilice la API).
 - Método: POST.
 - Ruta: /api/usuarios (o la ruta del endpoint a probar).
 - Marcar la opción **Use Body Data**.
 - Ingresar el contenido en formato JSON

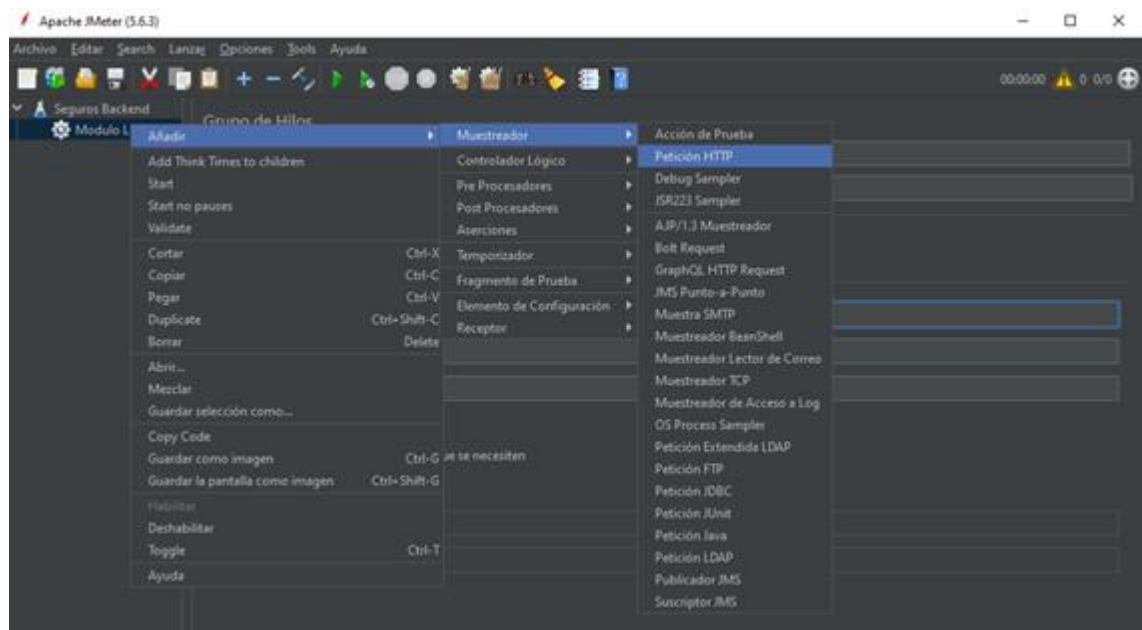


Ilustración 19 Agregación de petición HTTP



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025

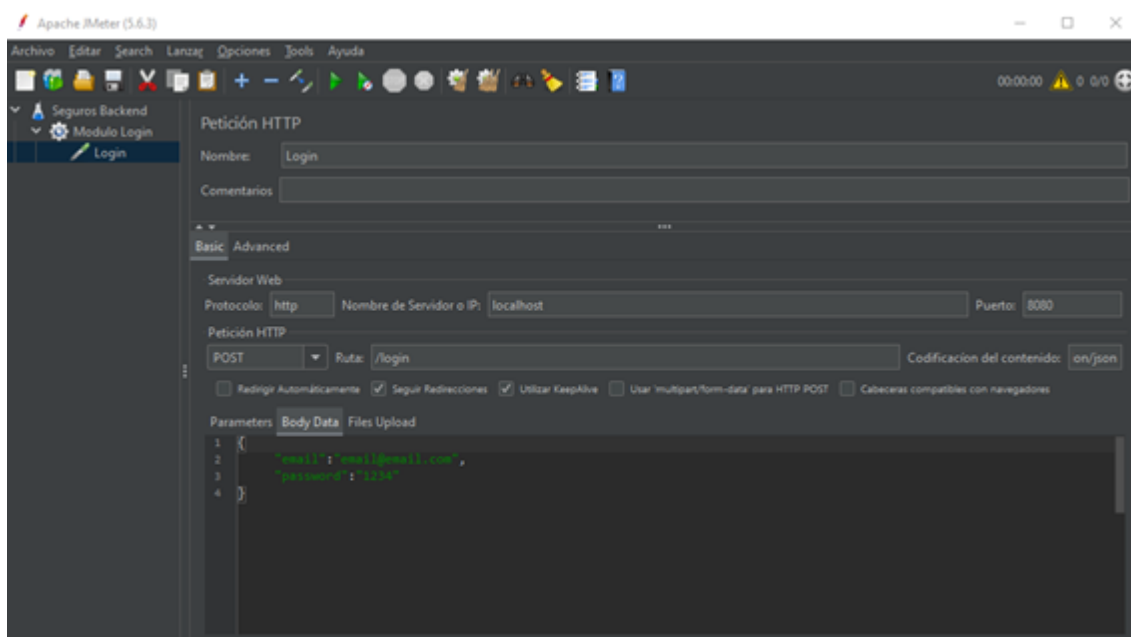


Ilustración 20 configuración de ruta, método y contenido de la petición

Configuración de Cabeceras HTTP

- Clic derecho sobre el **HTTP Request**.
- Seleccionar **Agregar** → **Config Element** → **HTTP Header Manager**.
- Agregar las siguientes cabeceras:

Nombre	Valor
Content-Type	application/json; charset=UTF-8
Authorization	Bearer [TOKEN]
Accept	application/json
Connection	keep-alive

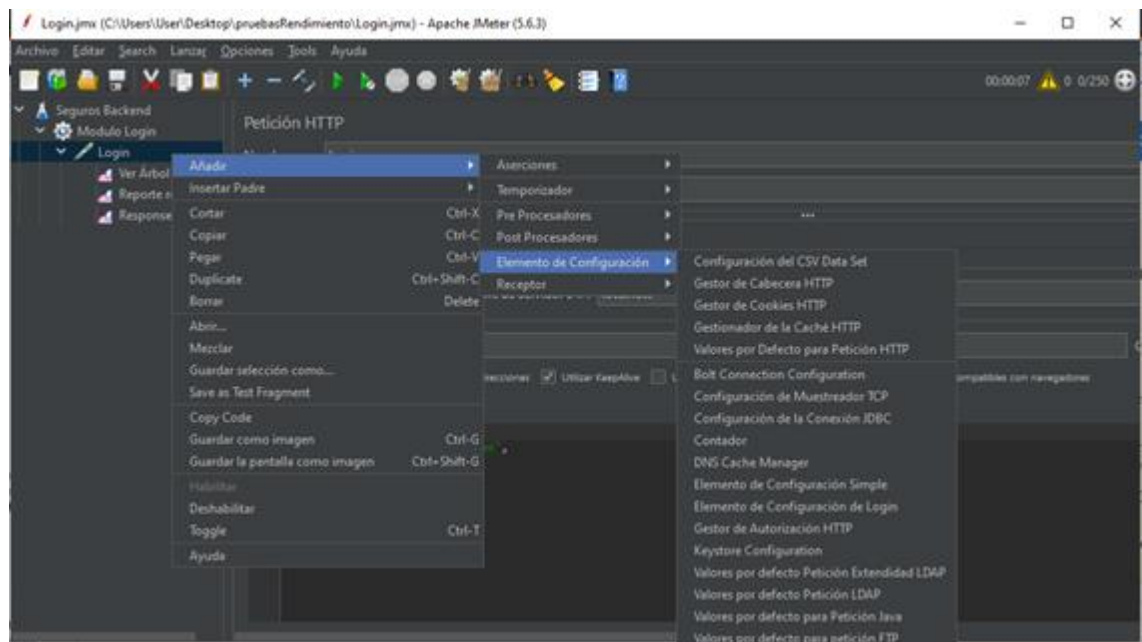


Ilustración 21 Agregación de contexto de cabecera para la petición

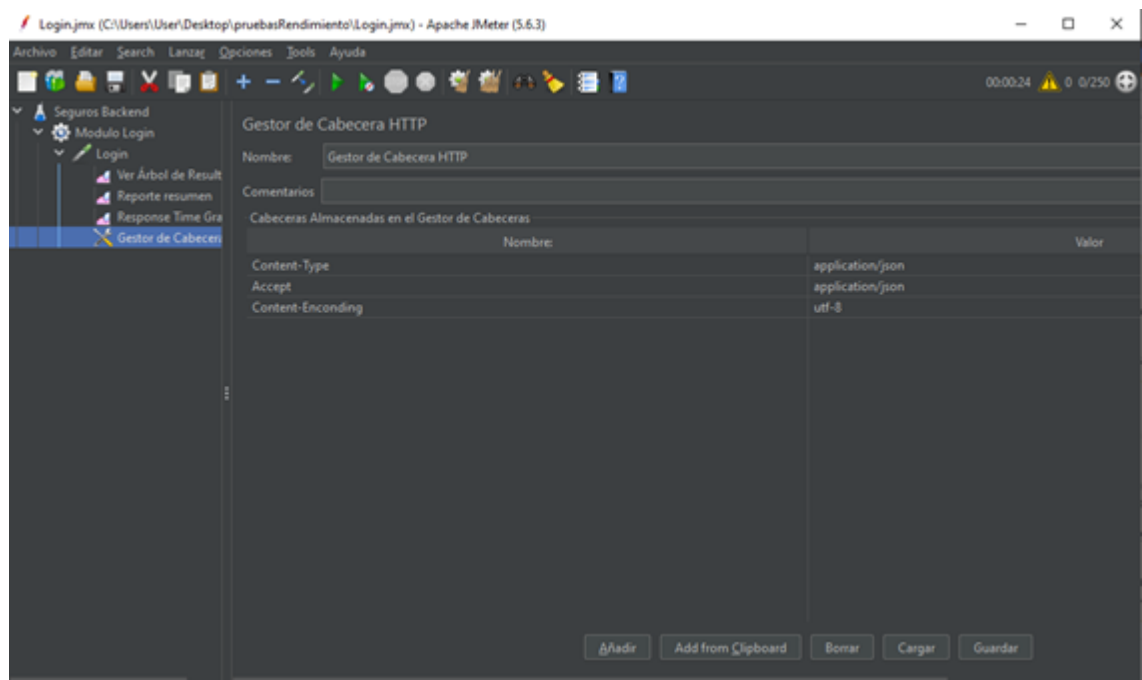


Ilustración 22 configuración de cabecera para tipo json

Agregar un Listener para visualizar resultados

- Clic derecho sobre el **Grupo de Hilos**.
- Seleccionar **Agregar** → **Listener** → **Ver Árbol de Resultados**.
- Este componente permitirá observar las respuestas de la API después de cada ejecución.

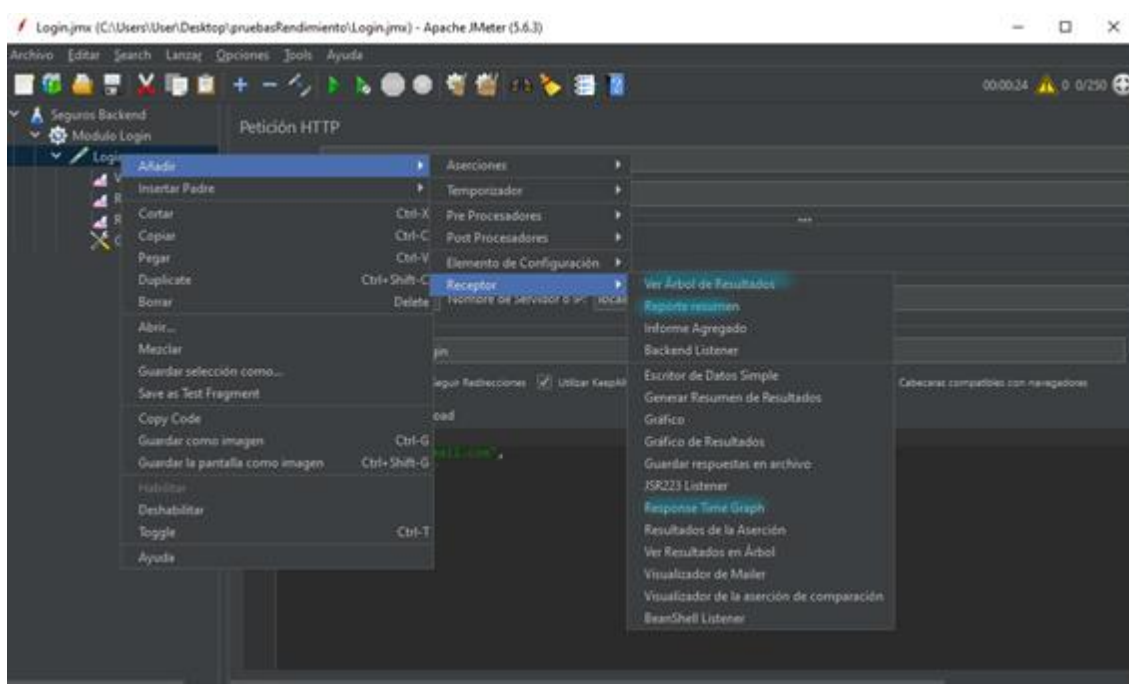
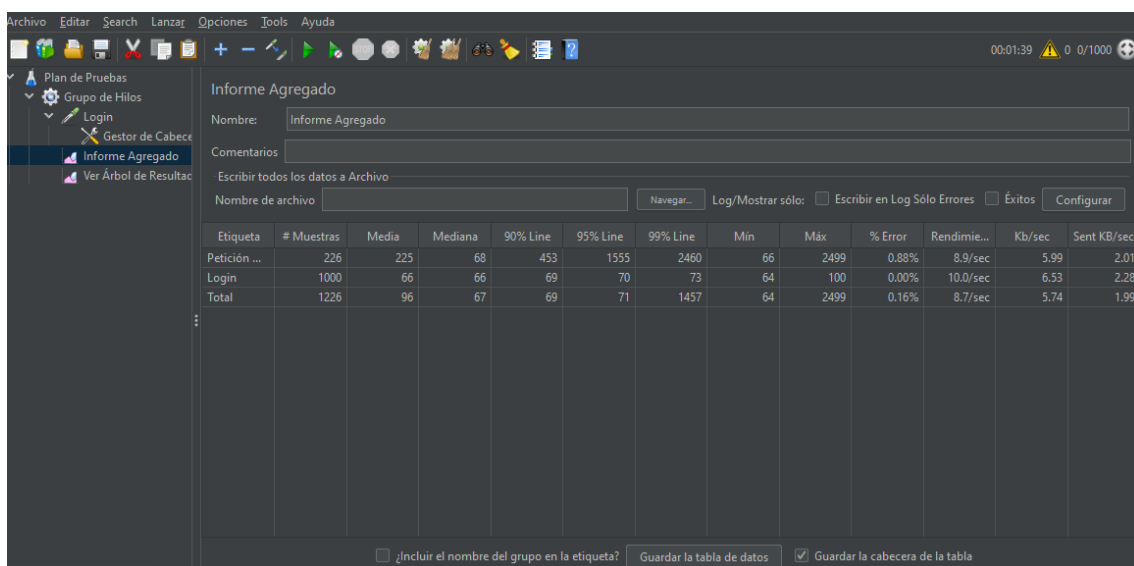


Ilustración 23 Inserción de receptores para informes

Ejecución de la Prueba

- Presionar el botón **Iniciar** (ícono de "Play" verde en la barra superior).
- Observar el resultado en el Listener:
 - Código de respuesta esperado: 200 OK o 201 Created.
 - Cuerpo de respuesta y cabeceras devueltas por el servidor.



The screenshot shows the 'Informe Agregado' window in Apache JMeter. It displays a table with performance metrics for the 'Login' test. The table includes columns for 'Etiqueta', '# Muestras', 'Media', 'Mediana', '90% Line', '95% Line', '99% Line', 'Min', 'Máx', '% Error', 'Rendimie...', 'Kb/sec', and 'Sent KB/sec'.

Etiqueta	# Muestras	Media	Mediana	90% Line	95% Line	99% Line	Min	Máx	% Error	Rendimie...	Kb/sec	Sent KB/sec
Petición ...	226	225	68	453	1555	2460	66	2499	0.88%	8.9/sec	5.99	2.01
Login	1000	66	66	69	70	73	64	100	0.00%	10.0/sec	6.53	2.28
Total	1226	96	67	69	71	1457	64	2499	0.16%	8.7/sec	5.74	1.99

Ilustración 24 Ejecución de las pruebas

Interpretación de Pruebas

En las pruebas de carga y rendimiento realizadas con Apache JMeter, se simularon un total de 1226 peticiones al endpoint de **login**, de las cuales 1000 correspondieron directamente a solicitudes de autenticación. El tiempo medio de respuesta fue de **66 ms**,



con un tiempo máximo de **100 ms** y sin errores, lo que indica un comportamiento estable bajo carga. Por otro lado, se observó que otras peticiones alcanzaron una media de **225 ms** y un tiempo máximo de **2499 ms**, presentando una tasa de error del **0.88%**, lo que podría reflejar cuellos de botella o puntos de saturación específicos. El rendimiento general del sistema fue de **8.7 peticiones por segundo**, lo que permite evaluar su capacidad de respuesta ante múltiples solicitudes concurrentes y detectar posibles mejoras en la eficiencia del backend.

Pruebas de seguridad

Descarga

1. Ir al sitio oficial: <https://www.zaproxy.org/download/>
2. Descargar el instalador adecuado según el sistema operativo:
 - a. Windows: .exe

Instalación

- Ejecutar el archivo .exe descargado y seguir el asistente.

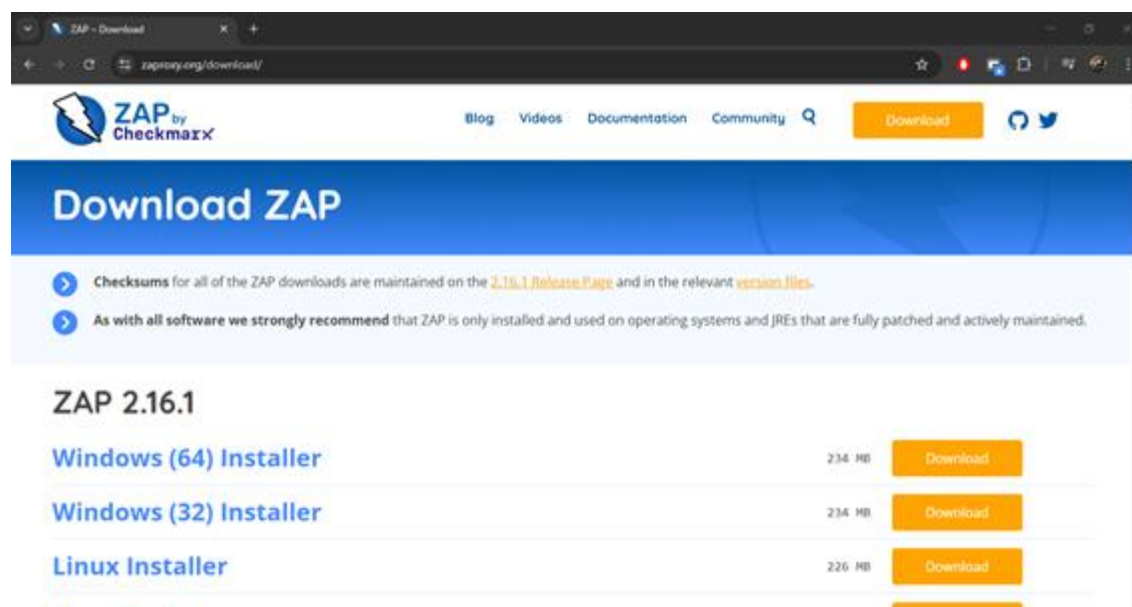


Ilustración 25 Página oficial de descarga de Zap

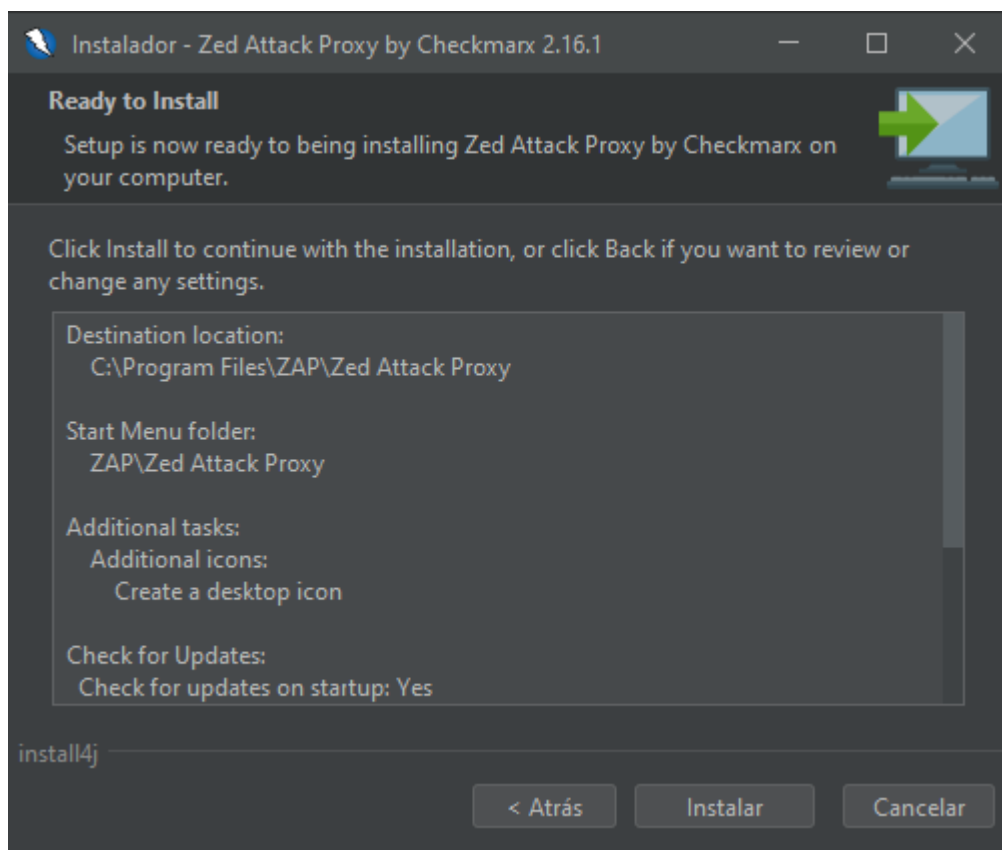


Ilustración 26 Instalación de Zap

Configurar OWASP ZAP como proxy

Por defecto, ZAP se configura para interceptar tráfico HTTP en:

- **Host:** localhost
- **Puerto:** 8080

Verifica esto en ZAP:

1. Ir a Tools -> Options -> Local Proxies
2. Confirmar que exista una entrada como:

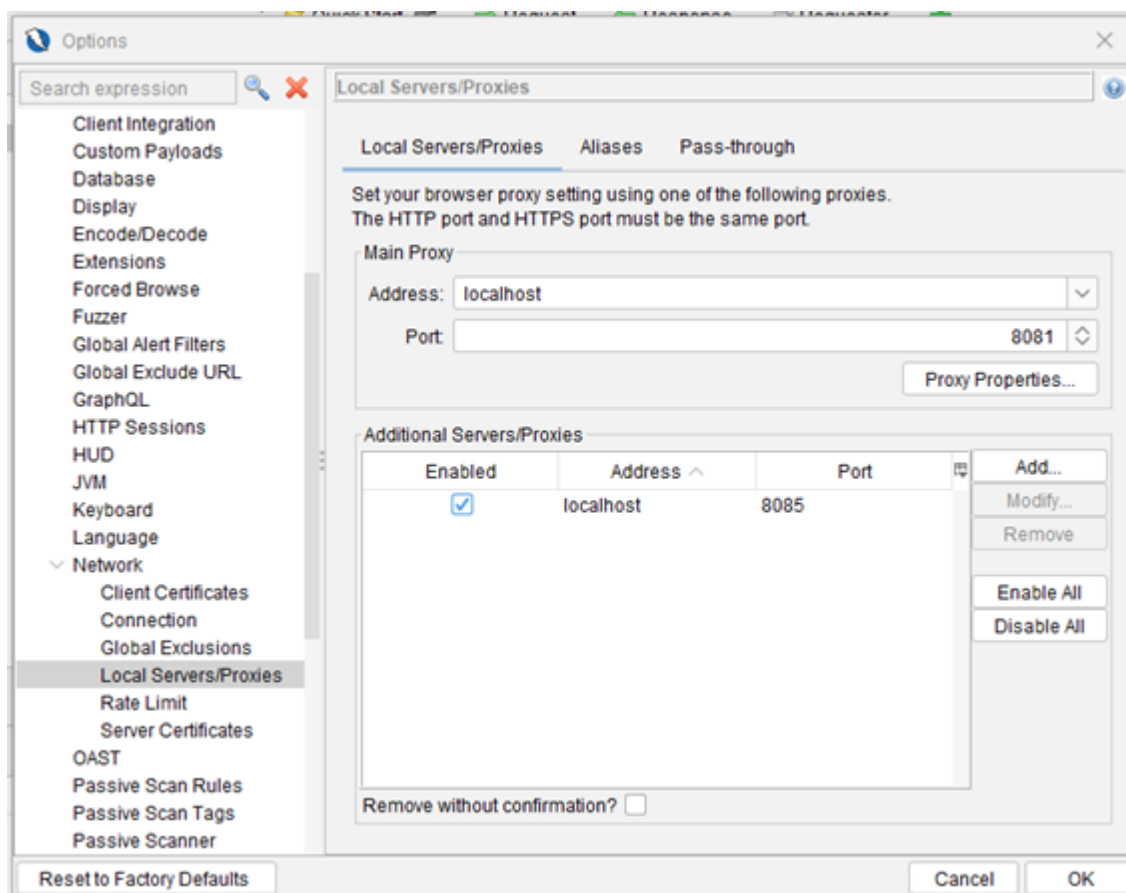


Ilustración 27 configuración de Proxy de zap

Configurar Postman para usar ZAP como proxy

1. Abrir Postman
2. Ir a Settings
3. En la pestaña **Proxy**:
 - a. Habilitar Use System Proxy
 - b. O configurar manualmente:
 - i. **Proxy Server:** localhost
 - ii. **Port:** 8080
 - iii. **Protocol:** HTTP

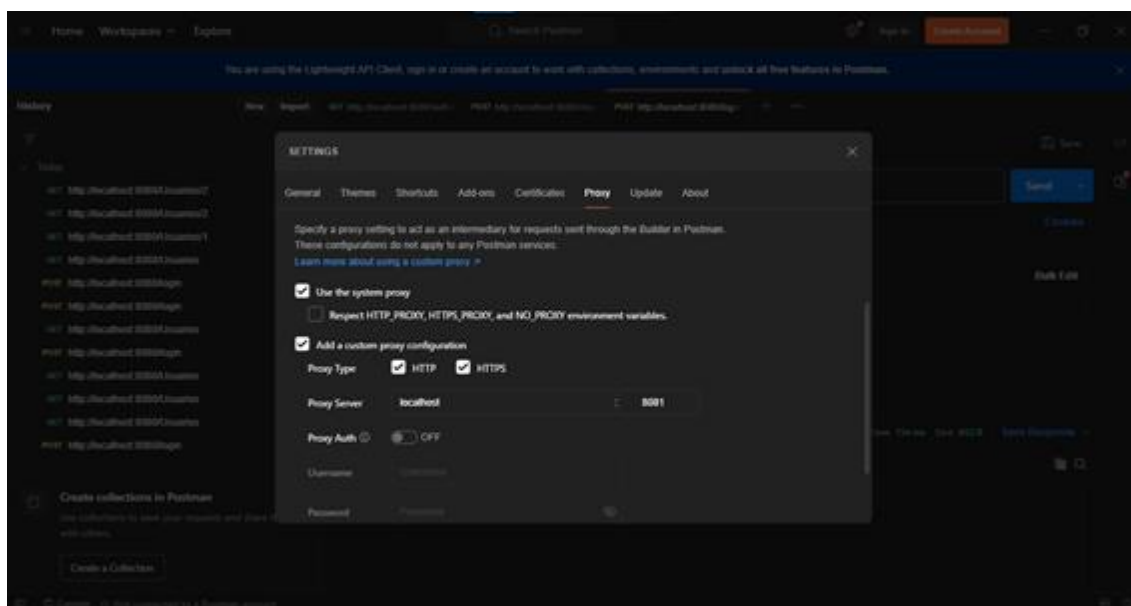


Ilustración 28 configuración de proxy en postman

Interceptar tráfico desde Postman

1. En ZAP, asegúrate de que la pestaña **Sites** esté vacía o limpia.
2. Desde Postman, realiza una petición HTTP a tu API o servidor local/remoto:
 - a. Ejemplo: GET <http://localhost:8081/usuarios>
3. La petición aparecerá en ZAP en la pestaña:
 - a. Sites

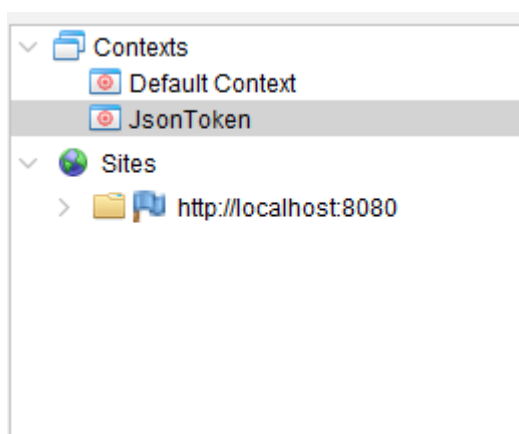


Ilustración 29 Sitios y peticiones realizadas desde postman en sites de zap

Análisis activo (sin autenticación)

1. Click derecho sobre el dominio en el árbol de Sites
2. Seleccionar **Attack** -> **Active Scan**
3. Dejar la configuración por defecto
4. Iniciar escaneo

ZAP enviará múltiples peticiones maliciosas para detectar:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



- Inyecciones
- XSS
- Headers inseguros
- Métodos HTTP inseguros, etc.

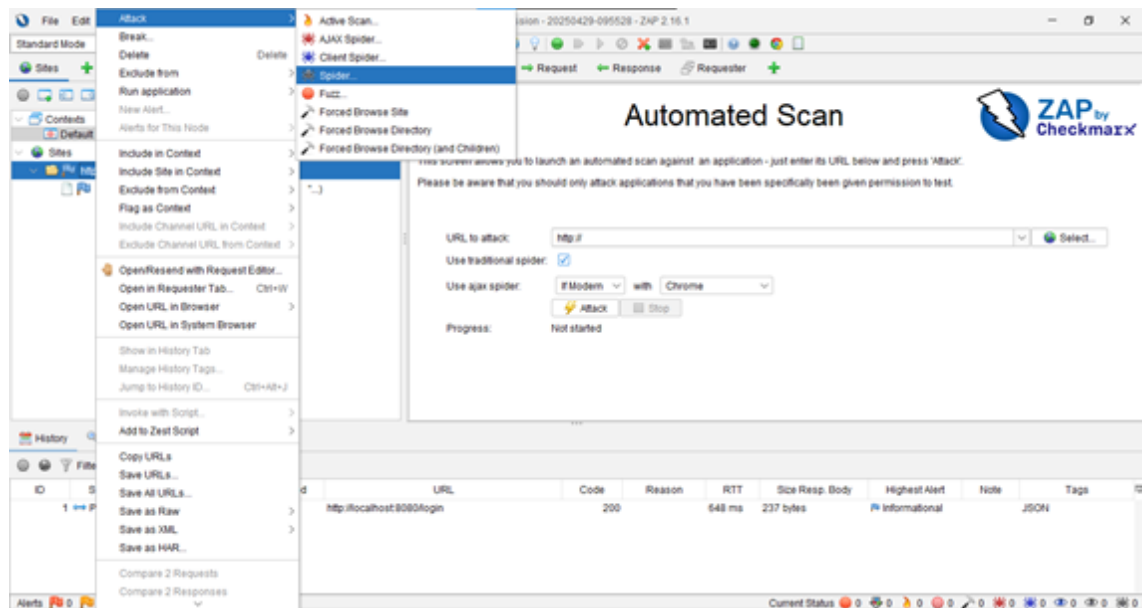


Ilustración 30 Ataque spider en la ruta del proyecto

ID	Req. Timestamp	Resp. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Header	Size Resp. Body
230	29/04/25 10:27:44	29/04/25 10:27:44	POST	http://localhost:8080/login	200	68 ms	413 bytes	237 bytes	
231	29/04/25 10:27:44	29/04/25 10:27:44	POST	http://localhost:8080/login	200	66 ms	413 bytes	237 bytes	
232	29/04/25 10:27:44	29/04/25 10:27:44	POST	http://localhost:8080/login	500	66 ms	391 bytes	91 bytes	
233	29/04/25 10:27:44	29/04/25 10:27:45	POST	http://localhost:8080/login	500	66 ms	391 bytes	91 bytes	
234	29/04/25 10:27:45	29/04/25 10:27:45	POST	http://localhost:8080/login	500	65 ms	391 bytes	91 bytes	
235	29/04/25 10:27:45	29/04/25 10:27:45	POST	http://localhost:8080/login	500	66 ms	391 bytes	91 bytes	
236	29/04/25 10:27:45	29/04/25 10:27:45	POST	http://localhost:8080/login	500	66 ms	391 bytes	91 bytes	
237	29/04/25 10:27:45	29/04/25 10:27:45	POST	http://localhost:8080/login	500	66 ms	391 bytes	91 bytes	
238	29/04/25 10:27:45	29/04/25 10:27:45	POST	http://localhost:8080/login	200	68 ms	413 bytes	237 bytes	
239	29/04/25 10:27:45	29/04/25 10:27:45	POST	http://localhost:8080/login	200	68 ms	413 bytes	237 bytes	
240	29/04/25 10:27:45	29/04/25 10:27:45	POST	http://localhost:8080/login	500	67 ms	391 bytes	91 bytes	
241	29/04/25 10:27:45	29/04/25 10:27:45	POST	http://localhost:8080/login	500	66 ms	391 bytes	91 bytes	
242	29/04/25 10:27:45	29/04/25 10:27:45	POST	http://localhost:8080/login	500	67 ms	391 bytes	91 bytes	
243	29/04/25 10:27:45	29/04/25 10:27:45	POST	http://localhost:8080/login	500	68 ms	391 bytes	91 bytes	
244	29/04/25 10:27:45	29/04/25 10:27:45	POST	http://localhost:8080/login	500	67 ms	391 bytes	91 bytes	

Ilustración 31 Resultados de las peticiones realizadas por zap

Revisar resultados

- Ir a la pestaña **Alerts**
- Ahí verás:
 - Vulnerabilidades detectadas
 - Severidad (Alta, Media, Baja)
 - Descripción y solución recomendada



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025

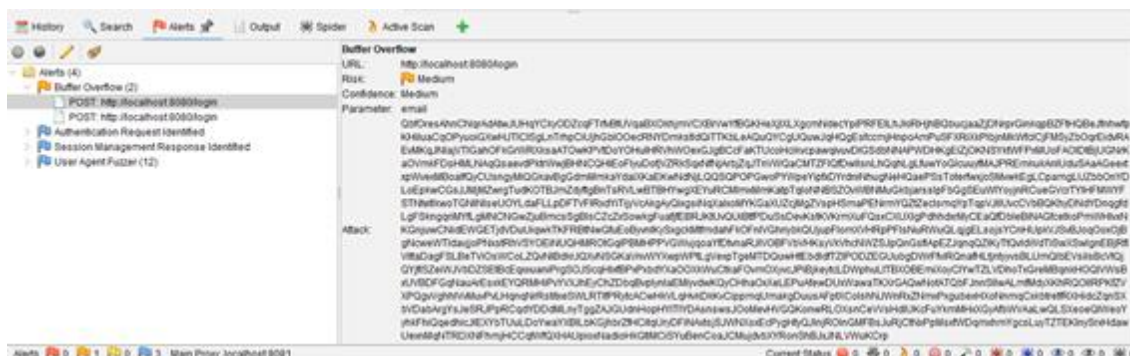


Ilustración 32 Alertas detectadas por zap

Interpretación de Pruebas

Durante las pruebas de seguridad realizadas con OWASP ZAP sobre el endpoint /login de la aplicación, se identificó una posible vulnerabilidad de **desbordamiento de búfer** en el parámetro email, lo que sugiere una falta de validación en la longitud o formato de los datos ingresados. Esta prueba se llevó a cabo mediante el envío de cadenas excesivamente largas con el fin de detectar fallos de manejo de entrada por parte del servidor. Además, se registraron otras alertas relacionadas con la gestión de sesiones, autenticación y pruebas con diferentes agentes de usuario, permitiendo evaluar el comportamiento de la aplicación frente a ataques automatizados y posibles debilidades en su seguridad.

FRONT-END

Pruebas funcionales

- Pruebas Unitarias

Las pruebas unitarias validan el comportamiento de unidades individuales de código (como funciones o métodos), asegurando que cada parte funcione correctamente de manera aislada. En este proyecto, las pruebas unitarias se realizan con “**Vitest**”, una herramienta moderna y rápida basada en Vite, que permite escribir y ejecutar pruebas de manera eficiente. Vitest ofrece integración con TypeScript, soporte para mocks, pruebas en paralelo y una experiencia de desarrollo ágil.

1er Paso:

Para realizar las pruebas unitarias se requiere ejecutar en la terminal la instalación de **Vitest** y las utilidades relacionadas con el siguiente comando:

```
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
```

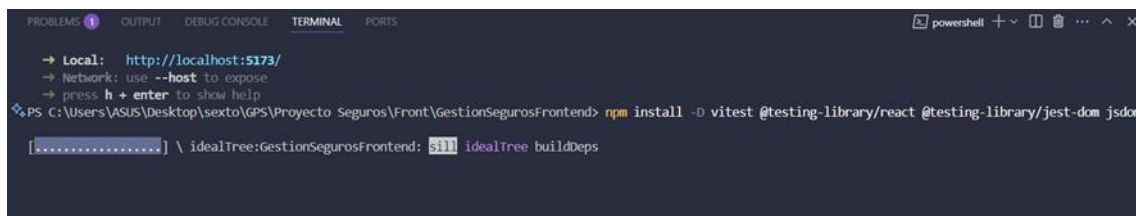


Ilustración 33 Instalación de Vitest

2do. Paso:



Como siguiente paso se realiza la configuración del archivo vite.config.js el cual sirve para habilitar y personalizar el entorno de pruebas con “Vitest” dentro del proyecto creado con Vite y React, simulando el entorno del navegador y cargue configuraciones previas necesarias para las pruebas.

```
vite.config.js > ...
1  import { defineConfig } from "vite";
2  import react from "@vitejs/plugin-react";
3  import tailwindcss from "@tailwindcss/vite";
4
5  // https://vite.dev/config/
6  export default defineConfig({
7    plugins: [react(), tailwindcss()],
8    test: {
9      environment: "jsdom", // Entorno para pruebas de componentes React
10     globals: true, // Habilita APIs globales como `describe`, `test`, `expect`
11     setupFiles: './src/Test/setupTests.js' // Archivo para configuraciones iniciales
12   },
13   resolve: {
14     alias: {
15       'jquery': 'jquery/src/jquery',
16     },
17   },
18   optimizeDeps: {
19     include: ["jquery", "datatables.net"],
20   },
21 });
22
```

Ilustración 34 Archivo de configuración vite.config.js

3er. Paso

Creación y configuración del entorno de pruebas antes de que se ejecuten los test, se debe importar una librería importante la cual agrega matchers personalizados a Jest/Vitest para hacer las aserciones más legibles y específicas al trabajar con el DOM.

```
src > Test > setupTests.js > vi.mock("primereact/inputnumber") callback > InputNumber
1  import React from "react";
2  import "@testing-library/jest-dom";
3  import { vi } from "vitest";
4
5
6  vi.mock("primereact/inputnumber", () => ({
7    InputNumber: ({ id, value, onChange, required, className, ...props }) =>
8      React.createElement("input", {
9        id,
10       value: value !== null && value !== undefined ? value : "",
11       onChange: (e) =>
12         onChange && onChange({ value: parseFloat(e.target.value) || 0 }),
13       required,
14       className,
15       type: "number",
16       ...props,
17     }),
18   }));
19
20  vi.mock("primereact/dropdown", () => ({
21    Dropdown: ({ id, value, options, onChange, placeholder, required, className, ...props }) =>
22      React.createElement(
23        "select",
24        {
25          id,
26          value: value || "",
27          onChange: (e) => onChange && onChange({ value: e.target.value }),
28          required,
29          className,
30          ...props,
31        },
32        React.createElement("option", { value: "", disabled: true }, placeholder || "Seleccione"),
33        options &&
34          options.map((opt) =>
35            React.createElement("option", { key: opt.value, value: opt.value }, opt.label)
36          )
37      )
38  });
```

Ilustración 35 Archivo setupTest.js



4to. Paso: Se crea una carpeta llamada `__test__`, dentro de esta carpeta crearan los archivos de pruebas unitarias que se realizarán por cada componente, en este caso se realizara la prueba en la página Login.jsx

```
1 import { vi } from 'vitest';
2 import { validateForm } from '../Login'; // Importamos la función a probar
3
4 describe('validateForm', () => {
5   // Mock de toast
6   const toastMock = {
7     current: {
8       show: vi.fn(), // Mockeamos la función show
9     },
10  };
11
12  beforeEach(() => {
13    // Limpiar el mock antes de cada prueba
14    toastMock.current.show.mockClear();
15  });
16
17  test('devuelve false y muestra advertencia si los campos están vacíos', () => {
18    const result = validateForm('', '', toastMock);
19
20    expect(result).toBe(false); // Debe devolver false
21    expect(toastMock.current.show).toHaveBeenCalledWith({
22      severity: 'warn',
23      summary: 'Campos incompletos',
24      detail: 'Por favor ingrese usuario y contraseña.',
25      life: 3000,
26    });
27  });
28
29  test('devuelve true y no muestra advertencia si los campos están completos', () => {
30    const result = validateForm('admin', '123456', toastMock);
31
32    expect(result).toBe(true); // Debe devolver true
33    expect(toastMock.current.show).not.toHaveBeenCalled(); // No se debe llamar a show
34  });
35
36  test('devuelve false y muestra advertencia si solo username está vacío', () => {
37    const result = validateForm('', '123456', toastMock);
38
39    expect(result).toBe(false);
40    expect(toastMock.current.show).toHaveBeenCalledWith({
41      severity: 'warn',
42      summary: 'Campos incompletos',
43      detail: 'Por favor ingrese usuario y contraseña.',
44      life: 3000,
45    });
46  });
47
48  test('devuelve false y muestra advertencia si solo password está vacío', () => {
49    const result = validateForm('admin', '', toastMock);
50
51    expect(result).toBe(false);
52    expect(toastMock.current.show).toHaveBeenCalledWith({
53      severity: 'warn',
54      summary: 'Campos incompletos',
55      detail: 'Por favor ingrese usuario y contraseña.',
56      life: 3000,
57    });
58  });
59 });
```

Ilustración 35 Creación de prueba unitaria para el login



Por último, se ejecuta la prueba unitaria desde la terminal con el comando:

```
npx vitest run src/pages/__test__/UnitariaLogin.test.jsx
```

```
PS C:\Users\ASUS\Desktop\sexto\GPS\Proyecto Seguros\Front\GestionSegurosFrontend> npx vitest run src/pages/__test__/UnitariaLogin.test.jsx
RUN v3.1.2 C:/Users/ASUS/Desktop/sexto/GPS/Proyecto Seguros/Front/GestionSegurosFrontend
✓ src/pages/__test__/UnitariaLogin.test.jsx (4 tests) 5ms
✓ validateForm > devuelve false y muestra advertencia si los campos están vacíos 3ms
✓ validateForm > devuelve true y no muestra advertencia si los campos están completos 0ms
✓ validateForm > devuelve false y muestra advertencia si solo username está vacío 0ms
✓ validateForm > devuelve false y muestra advertencia si solo password está vacío 0ms
Test Files 1 passed (1)
Tests 4 passed (4)
Start at 06:25:44
Duration 2.39s (transform 109ms, setup 316ms, collect 388ms, tests 5ms, environment 1.24s, prepare 171ms)
PS C:\Users\ASUS\Desktop\sexto\GPS\Proyecto Seguros\Front\GestionSegurosFrontend>
```

Ilustración 36 Ejecución de prueba unitaria.

Interpretación de resultados

Los resultados de la ejecución de `npx vitest run src/pages/\_\_test\_\_/UnitariaLogin.test.jsx` muestran que las 4 pruebas unitarias de la función `validateForm` en el archivo `UnitariaLogin.test.jsx` pasaron exitosamente en 5ms (3ms para "campos vacíos", 0ms para las otras tres), verificando que la función devuelve `false` y muestra una advertencia cuando los campos están vacíos o uno de ellos lo está, y `true` sin advertencia cuando están completos, con un tiempo total de ejecución de 2.39s incluyendo configuración.

La prueba pasó sin errores ni fallos, lo que confirma que el comportamiento esperado del componente está funcionando correctamente. Esto demuestra que la lógica implementada cumple con el requisito funcional y que la herramienta de pruebas (Vitest) está correctamente configurada y operativa.

- Pruebas de Integración

Una prueba de integración verifica cómo interactúan varios componentes o módulos entre sí dentro de una funcionalidad real del sistema. En el caso del login, permite asegurarse de que la interfaz de usuario, el manejo de estado, y la comunicación con el backend (API) funcionan juntos como se espera, simulando un escenario real de uso, pero sin depender de un servidor externo gracias al uso de mocks.

Se configura la simulación del acceso a la ApiREST mockeándolo, luego se renderiza el componente y se utiliza fireEvent para simular que el usuario escribe su correo y contraseña y luego envía el formulario. Y por último se espera la respuesta del servidor mostrando un mensaje de éxito.



```
Login.test.jsx M X Login.jsx
src > pages > __test__ > Login.test.jsx > ...
60 describe('Login - flujo exitoso', () => {
61   beforeEach(() => {
62     mockNavigate.mockClear()
63   })
64
65   test('loguea exitosamente y redirige al rol adecuado', async () => {
66     render(
67       <BrowserRouter>
68         <Login />
69       </BrowserRouter>
70     )
71
72     // Completar los campos
73     fireEvent.change(screen.getByLabelText(/Nombre de usuario/i), {
74       target: { value: 'admin' },
75     })
76     fireEvent.change(screen.getByLabelText(/Contraseña/i), {
77       target: { value: '123456' },
78     })
79
80     // Clic en botón
81     fireEvent.click(screen.getByRole('button', { name: /Iniciar Sesión/i }))
82
83     // Esperar redirección (timeout + decode + navigate)
84     await waitFor(() => {
85       expect(localStorage.getItem('jwtToken')).toBe('fake-jwt-token')
86       expect(mockNavigate).toHaveBeenCalledWith('/Home')
87     }, { timeout: 4000 })
88   })
89 })
90
```

Ilustración 37 Configuración de prueba de integración.

Finalmente se realiza la prueba de integración con el comando.

npx vitest run src/pages/__test__/LoginIntegracion.test.jsx

```
PS C:\Users\ASUS\Desktop\sexto\GPS\Proyecto Seguros\Front\GestionSegurosFrontend> npx vitest run src/pages/__test__/LoginIntegracion.test.jsx
RUN v3.1.2 C:\Users\ASUS\Desktop\sexto\GPS\Proyecto Seguros\Front\GestionSegurosFrontend
✓ src/pages/__test__/LoginIntegracion.test.jsx (2 tests) 3208ms
✓ Login component > muestra advertencia si los campos están vacíos 145ms
✓ Login - flujo exitoso > loguea exitosamente y redirige al rol adecuado 3062ms

Test Files 1 passed (1)
Tests 2 passed (2)
Start at 06:36:59
Duration 4.73s (transform 94ms, setup 197ms, collect 277ms, tests 3.21s, environment 639ms, prepare 141ms)
```

Ilustración 38 Resultado de la prueba de integración.

Interpretación de resultados

El resultado de la prueba de integración es que verifica que el flujo de login sea exitoso, es decir que, al ingresar credenciales válidas, el usuario se logue correctamente y sea redirigido a una página adecuada según su rol

Pruebas no funcionales

- Pruebas de rendimiento

El objetivo principal de estas pruebas es simular y evaluar aspectos del sistema con cómo funciona el sistema es decir las pruebas de rendimiento, el performance y la accesibilidad que tiene la página web. Estas se realizarán con **Lighthouse**.

Pasos para realizar las pruebas.

- Abrir el sitio web Google Chrome junto con la dirección local en la que corre el proyecto. En el caso del proyecto este corre en la dirección: <http://localhost:5173/>
- Presionando F12 abrimos el inspector de código o línea para desarrolladoras.
- Ir a la pestaña de “Lighthouse”



- Seleccionamos el tipo de prueba a realizar en este caso: desktop. En categorías elegimos: Accessibility, Performance.
- Ejecutamos e análisis haciendo clic en “Generate report”. Mostrándose un informe detallado con puntajes y recomendaciones.

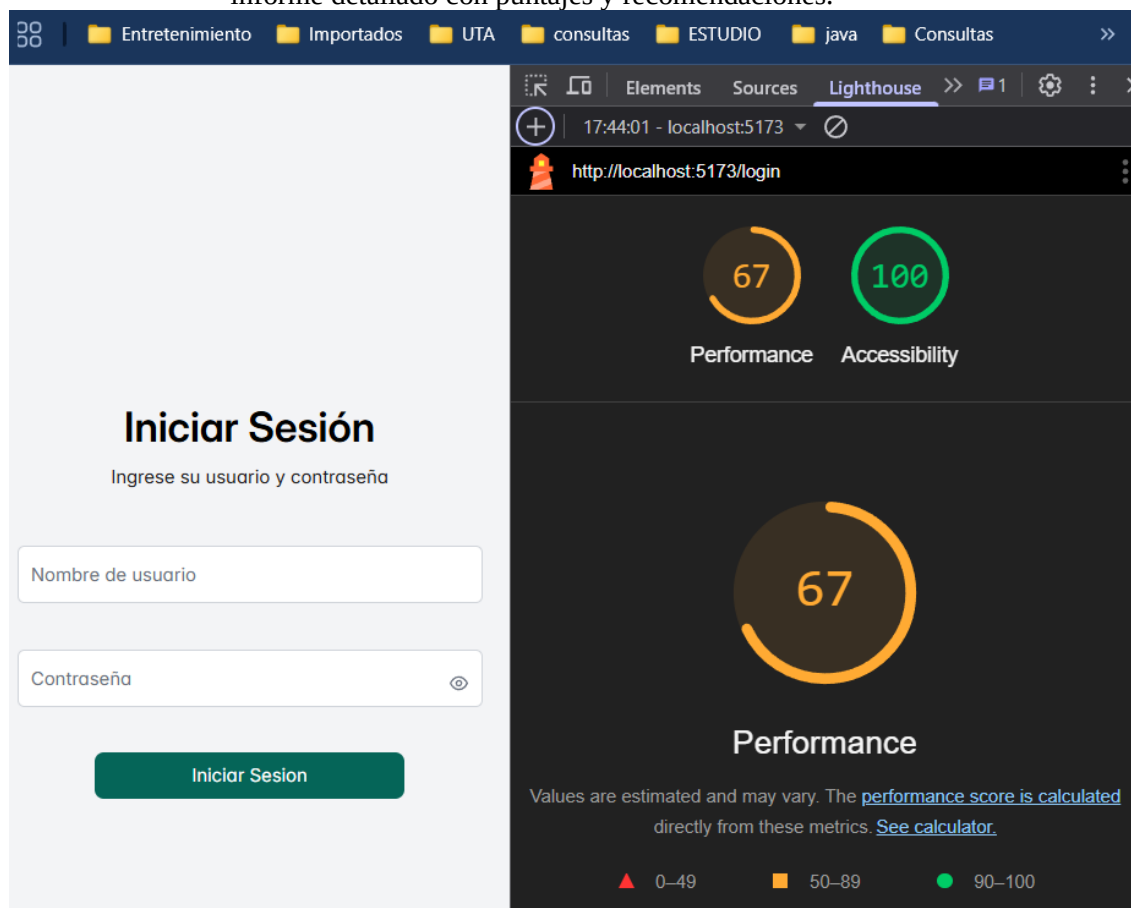


Ilustración 39 Resultados de las pruebas no funcionales.

Interpretación de resultados

Los resultados obtenidos sobre la página de inicio de sesión. Se obtuvo un puntaje de rendimiento de un 67 y en accesibilidad de 100. Esto garantiza que la experiencia de usuario será óptima y que la aplicación cumple con estándares de calidad.

- Pruebas de carga

Las pruebas de carga son un tipo de prueba de rendimiento que se realizan con el objetivo de medir el comportamiento de una aplicación o sistema bajo una carga específica, normalmente con un número elevado de usuarios o transacciones concurrentes.

K6 es una herramienta de código abierto especializada en pruebas de carga, es una herramienta sencilla, poderosa y flexible para garantizar que el frontend pueda manejar el tráfico y las demandas del mundo real de manera eficiente.

Pasos para realizar las pruebas de carga con k6

- Instalar **chocolatey** para luego instalar **k6**, herramienta que será crucial para realizar las pruebas de rendimiento. Esta se puede descargar directamente desde la siguiente dirección <https://chocolatey.org/install>. Todo el proceso se realiza mediante la Power Shell ejecutándose como administrador



- Una vez instalada chocolatey. Se procede a realizar la instalación de k6 con el comando `choco install K6`.

Una vez instalado todas las herramientas necesarias para las pruebas de carga. Se debe crear una carpeta llamada **k6-tests**.

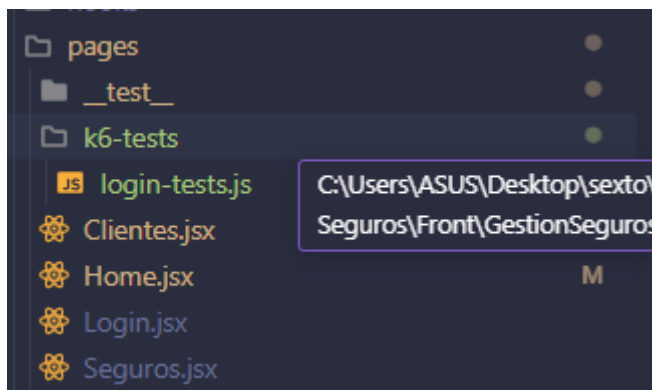


Ilustración 40 Carpeta creada para realizar los tests con k6

Dentro de la carpeta se crea el script que simulará múltiples solicitudes al endpoint. Esta prueba está aplicada únicamente a la parte de login en el proyecto de Gestión de Seguros.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

// Configuración de la prueba
export const options = {
  vus: 10, // Número de usuarios virtuales
  duration: '30s', // Duración de la prueba
  thresholds: {
    http_req_duration: ['p(95)<500'], // 95% de las solicitudes deben tardar menos de 500ms
    http_req_failed: ['rate<0.01'], // Menos del 1% de las solicitudes deben fallar
  },
};

// Datos de prueba (credenciales de un usuario válido)
const BASE_URL = 'http://localhost:8080'; // Cambia esto por la URL de tu backend
const LOGIN_URL = `${BASE_URL}/login`;
const USER_CREDENTIALS = {
  email: 'email@email.com', // Cambia por un email válido
  password: '1234', // Cambia por una contraseña válida
};

export default function () {
  // Enviar solicitud POST al endpoint /login
  const res = http.post(LOGIN_URL, JSON.stringify(USER_CREDENTIALS), {
    headers: { 'Content-Type': 'application/json' },
  });

  // Verificar que la respuesta sea correcta
  check(res, {
    'status is 200': (r) => r.status === 200,
    'token received': (r) => r.json('token') !== undefined,
  });

  // Simular una pausa entre solicitudes (pensar como un usuario real)
  sleep(1);
}
```

Ilustración 41 Script para pruebas de carga login.

Ejecución del script desde la terminal con el comando:
`k6 run src/pages/k6-tests/login-tests.js`



```
PS C:\Users\ASUS\Desktop\sexto\GPS\Proyecto Seguros\Front\GestionSegurosFrontend> k6 run src/pages/k6-tests/login-tests.js

Grafana
K6

execution: local
script: src/pages/k6-tests/login-tests.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 1m0s max duration (incl. graceful stop):
  * default: 10 looping VUs for 30s (gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<500' p(95)=89.74ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

TOTAL RESULTS

checks_total.....: 560      18.132624/s
checks_succeeded.....: 100.00% 560 out of 560
checks_failed.....: 0.00%    0 out of 560

✓ status is 200
✓ token received

HTTP
http_req_duration.....: avg=101.7ms min=69.73ms med=80.5ms max=671.36ms p(90)=86.36ms p(95)=89.74ms
{ expected:response:true }.....: avg=101.7ms min=69.73ms med=80.5ms max=671.36ms p(90)=86.36ms p(95)=89.74ms
http_req_failed.....: 0.00%    0 out of 280
http_reqs.....: 280      9.066312/s

EXECUTION
iteration_duration.....: avg=1.1s   min=1.07s   med=1.08s   max=1.68s   p(90)=1.08s   p(95)=1.09s
iterations.....: 280      9.066312/s
vus.....: 10      min=10      max=10
vus_max.....: 10      min=10      max=10

NETWORK
data_received.....: 174 kB 5.6 kB/s
data_sent.....: 52 kB 1.7 kB/s

running (0m30.9s), 00/10 VUs, 280 complete and 0 interrupted iterations
default ✓ [=====] 10 VUs 30s
```

Ilustración 42 Resultado de prueba k6 de carga.

Interpretación de resultados

Resultados obtenidos de la prueba de carga realizada al login, esta prueba fue configurada para que simule el ingreso de 10 usuarios virtuales durante 30 segundos. Se realizaron 280 solicitudes http teniendo un total de 100% de verificaciones pasaron, lo que las solicitudes fueron exitosas, sin fallos algunos.

Análisis de código estático

Para las pruebas de análisis de código estático se utiliza la herramienta **ESLint**, es una herramienta que ayuda a mantener un estilo de código consistente y libre de errores comunes examinando el código sin ejecutarlo. Mejorando la calidad y mantenibilidad del código

Pasos para realizar las pruebas con ESLint

- Instalar ESLint y las dependencias necesarias con el comando:
npm install eslint eslint-plugin-react eslint-plugin-jsx-a11y --save-dev

```
PS C:\Users\ASUS\Desktop\sexto\GPS\Proyecto Seguros\Front\GestionSegurosFrontend> npx eslint --init
You can also run this command directly using 'npm init @eslint/config@latest'.
@eslint/create-config: v1.8.1

✓ What do you want to lint? · javascript
✓ How would you like to use ESLint? · problems
✓ What type of modules does your project use? · esm
✓ Which framework does your project use? · react
✓ Does your project use TypeScript? · no / yes
✓ Where does your code run? · browser
The config that you've selected requires the following dependencies:

eslint, @eslint/js, globals, typescript-eslint, eslint-plugin-react
✓ Would you like to install them now? · No / Yes
✓ Which package manager do you want to use? · npm
Installing...
```

Ilustración 43 Instalación de dependencias para ESLint



- Inicialización de la configuración de ESLint con el comando:
npx eslint --init
Automáticamente se crea el archivo eslint.config.js

```
1 import js from '@eslint/js';
2 import globals from 'globals';
3 import tseslint from 'typescript-eslint';
4 import pluginReact from 'eslint-plugin-react';
5 import { defineConfig } from 'eslint/config';
6
7 export default defineConfig([
8   {
9     files: ['**/*.js,mjs,cjs,ts,jsx,tsx'],
10    plugins: { js },
11    extends: ['js/recommended'],
12  },
13  {
14    files: ['**/*.js,mjs,cjs,ts,jsx,tsx'],
15    languageOptions: {
16      globals: {
17        ...globals.browser,
18        ...globals.vitest, // Para pruebas con Vitest
19      },
20    },
21  },
22  tseslint.configs.recommended,
23  {
24    ...pluginReact.configs.flat.recommended,
25    settings: { react: { version: 'detect' } },
26    rules: {
27      'react/react-in-jsx-scope': 'off',
28      'react/prop-types': 'off',
29    },
30  },
31 ]);
```

Ilustración 44 Archivo de configuración ESLint.

- Ejecución de análisis de código estático del proyecto con el comando:
npx eslint src

```
PS C:\Users\ASUS\Desktop\sexto\GPS\Proyecto Seguros\Front\GestionSegurosFrontend> npx eslint src
C:\Users\ASUS\Desktop\sexto\GPS\Proyecto Seguros\Front\GestionSegurosFrontend\src\pages\Clientes.jsx
24:5  error  'estados' is assigned a value but never used  @typescript-eslint/no-unused-vars

X1 problem (1 error, 0 warnings)

PS C:\Users\ASUS\Desktop\sexto\GPS\Proyecto Seguros\Front\GestionSegurosFrontend> npx eslint src
PS C:\Users\ASUS\Desktop\sexto\GPS\Proyecto Seguros\Front\GestionSegurosFrontend>
```

Ilustración 45 Respuesta de análisis de pruebas estáticas

Interpretación de resultados

En la primera ejecución de la prueba estáticas de código se encontró un error, este se debía a que una variable estaba asignada pero que nunca fue utilizada. Se eliminó la variable y se volvió a correr la prueba, entonces ahí ya no se encontró más fallos en el código dando como respuesta positiva de la prueba.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



All files GestionSegurosFrontend/src/assets/icons/Components

100% Statements 189/189 100% Branches 5/5 100% Functions 5/5 100% Lines 189/189

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements	Branches	Functions	Lines
IconoCerrarSesion.jsx		100%	33/33	100%	1/1
IconoClientes.jsx		100%	13/13	100%	1/1
IconoCrear.jsx		100%	26/26	100%	1/1
IconoPerfil.jsx		100%	16/16	100%	1/1
SegurosIcon.jsx		100%	16/16	100%	1/1
index.js		100%	5/5	100%	0/0

Ilustración 46 Cobertura de código en componentes de íconos de React

All files GestionSegurosFrontend/src/components

89.87% Statements 213/237 95.34% Branches 41/43 85.71% Functions 18/21 89.87% Lines 213/237

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements	Branches	Functions	Lines
AccionesTemplate.jsx		100%	31/31	100%	9/9
Boton.jsx		100%	31/31	100%	2/2
DataTableGenerico.jsx		100%	57/57	100%	11/11
FormularioGenerico.jsx		71.08%	59/83	86.66%	13/15
ModalFormulario.jsx		100%	26/26	100%	3/3
ProtectedRoute.jsx		100%	9/9	100%	3/3

Ilustración 47 Cobertura de código en componente reutilizables React

All files GestionSegurosFrontend/src/pages

83.63% Statements 639/764 69.44% Branches 50/72 84.61% Functions 22/26 83.63% Lines 639/764

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements	Branches	Functions	Lines
Clientes.jsx		60.25%	144/239	57.14%	4/7
Home.jsx		89.92%	116/129	60.71%	17/28
Login.jsx		87.02%	114/131	80%	12/15
Seguros.jsx		100%	265/265	77.27%	17/22

Ilustración 48 Cobertura de código en páginas hechas en React

2.8 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☒ Trabajo en equipo
- ☒ Comunicación asertiva
- ☒ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad



2.9 Conclusiones

La implementación de pruebas permitió validar la calidad del sistema desde diferentes enfoques. Las pruebas unitarias e integración desarrolladas con JUnit aseguraron la correcta ejecución de la lógica interna de los componentes y su adecuada interacción dentro del contexto de Spring Boot, utilizando una base de datos H2 en memoria que garantizó un entorno controlado y replicable para cada prueba. Este enfoque facilitó la detección temprana de errores y contribuyó a mantener una arquitectura modular y confiable.

Por otro lado, las pruebas no funcionales ejecutadas con Apache JMeter permitieron evaluar el comportamiento del sistema ante distintos volúmenes de carga, midiendo su rendimiento, tiempo de respuesta y estabilidad bajo escenarios de estrés. A su vez, la integración de OWASP ZAP como herramienta de análisis de seguridad hizo posible identificar vulnerabilidades presentes en los endpoints expuestos, fortaleciendo así los aspectos de protección del sistema frente a posibles amenazas externas.

La documentación de cada una de las pruebas, desde su configuración hasta su ejecución, permitió elaborar una guía práctica que puede ser reutilizada en futuros desarrollos. La experiencia adquirida durante el proceso demuestra la importancia de incorporar prácticas de validación continua dentro del ciclo de desarrollo, ya que aportan valor al producto final y refuerzan la confianza en su funcionamiento. Este manual representa un recurso clave para estandarizar el uso de herramientas de prueba dentro del equipo y en proyectos similares, tanto académicos como profesionales.

2.10 Recomendaciones

- Mantener la automatización de pruebas en el ciclo de desarrollo: Integrar las pruebas unitarias y de integración como parte del proceso de CI/CD asegura la detección continua de fallos.
- Actualizar y ampliar la cobertura de pruebas: Es recomendable ampliar los casos de prueba tanto unitarios como de integración para cubrir excepciones, flujos alternativos y validaciones de reglas de negocio.
- Monitorear el rendimiento en entornos reales: Si bien JMeter permite simular escenarios de carga, se recomienda complementar con pruebas en ambientes más cercanos a producción para obtener métricas más precisas.

2.11 Referencias bibliográficas

Insertar las referencias bibliográficas empleadas aplicando la norma IEEE.

2.12 Anexos

